INDEX

NAME: NEHA. S YR STD.: 3 DEPT SEC.: CSD ROLL NO.: SUB.: GRAPHICS & MULTIMEDIA OBSERVATION.

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	21.07.26	BRESENHAM'S LINE DRAWING ALGORITHM FOR $\|m\|<1$	3 - 7	
2.	21.7.26	BRESENHAM'S LINE DRAWING ALGORITHM FOR CIRCLE	8 - 13	
3	21.7.26	BRESENHAM'S LINE DRAWING ALGORITHM FOR ELLIPSE	14 - 19.	
4.	04.08.26.	IMPLEMENTATION OF DDA LINE DRAWING ALGORITHM.	20 - 23	
5.	04.08.26	OBJECT DRAWING USING BRESENHAM'S ALGORITHM.		
6.	11.08.26	DRAW A RECTANGLE BRESENHAM'S ALGORITHM		
7.	11.08.26	DRAW A TRAFFIC SIGNAL USING BRESENHAM'S ALGORITHM.		
8.	25.8.26.	DRAW A SMILEY FACE USING BRESENHAM'S ALGORITHM		
9.	25.8.26	DRAW THE OLYMPICS LOGO USING BRESENHAM'S ALGORITHM		

GKC1025602 G

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
10.	25.8.26	DRAW A BUTTERFLY USING BRESENHAM'S ALGORITHM		
11.	8.9.26.	IMPLEMENT SCAN CONVERSION OF POLYGON AND USE FLOODFILL ALGORITHM.		
12.	22.9.2026	2D TRANSFORMATION TRIANGLE		
13	22.9.2026	2D TRANSFORMATION RECTANGLE.		
14.	22.9.26.	POLYGON CLIPPING.		
15.	29.9.26	3D TRANSFORMATION CUBE		
16.	29.9.26	3D TRANSFORMATION PYRAMID.		
17.	6.10.26	CREATE & RENDER 3D OBJECTS.		
18.	6.10.26	CREATE A MULTIMEDIA APPLICATION THAT INTEGRATE IMAGE, AUDIO & VIDEO.		
19.	6.10.26	CAPTURE AUDIO / VIDEO USING WEBCAM & MICROPHONE.		

completed

RAJALAKSHMI
ENGINEERING COLLEGE

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME ..NEHA.....S.....

ACADEMIC YEAR .2026-27. SEMESTER .....V.....BRANCH....CSD....

UNIVERSITY REGISTER No. 211624170103 9

Certified that this is the bonafide record of work done by the above student in the

IT23E31
GRAPHICS AND MULTIMEDIA..... Laboratory during the year 2026 - 2027

12/10/26

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on .....

External Examiner

Internal Examiner

20.07.26

EXPERIMENT : 1

BRESENHAM'S LINE DRAWING ALGORITHM FOR $|m|<1$

AIM:

To implement Bresenham's Line drawing algorithm for $|m|<1$ using python Programming.

PROGRAM:

```
import matplotlib.pyplot as plt

def bresenham_line(x1, y1, x2, y2):
    Points = []
    dx = x2 - x1
    dy = y2 - y1
    P = 2 * dy - dx
    x = x1
    y = y1
    Points.append((x,y))
```

```
while x<x2 :
    x = x+1
    if p<0 :
        p = p+2 * dy
    else :
        y = y+1
        p = p+2 * dy -2 *dx
    Points. append ((x,y))
return points

x1 = int (input ("Enter x1 : "))
y1 = int (input ("Enter y1 : "))

x2 = int (input ("Enter x2 : "))
y2 = int (input ("Enter y2 : "))

Points = bresenham_line (x1, y1, x2, y2)

Print ("Points generated : ")
for p in points :
    Print (p)

x_values = [p[0] for p in points]
y_values = [p[1] for p in points]

Plt. plot (x_values, y_values, marker = "o")
Plt. ptitle ("Bresenham Line Drawing Algo ")
Plt. xlabel ("x-axis")
Plt. ylabel ("y-axis")
```

```
Plt.grid()
Plt.show()
```

21/7/20

RESULT:

Hence implemented Bresenham's Line Drawing Algorithm for $|m| < 1$ using Python programming.

Input :

$x_1 = 3$

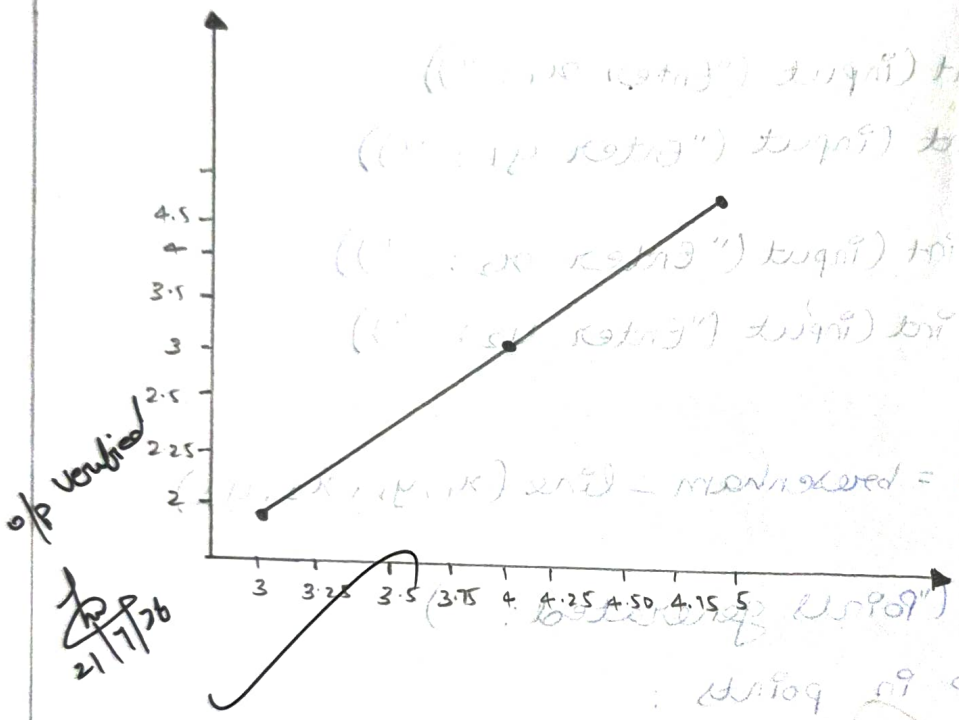
$y_1 = 2$

$x_2 = 5$

$y_2 = 4$.

Output :

Points generated : (3,2), (4,3), (5,4)



o/p verified

21/7/26

21/7/26

EXPERIMENT - 2

BRESENHAM'S LINE DRAWING ALGORITHM FOR CIRCLE.

AIM:

To implement Breserham's Line Drawing algorithm for circle using python program.

PROGRAM:

```
import matplotlib.pyplot as plt
def breserham_circle (xc, yc, r):
    Points =[]
    x=0
    y=r
    P = 3-2 * r
    def plot-circle_points (cx, cy, px, py):
        Points.append((cx + px, cy+py))
        Points.append((cx-px, cy +py))
        Points.append((cx +px, cy-py))
        Points.append((cx-px, cy- py))
        Points.append((cx + py, cy + px))
        Points.append((cx+py, cy-px))
        Points.append((cx-py, cy - px))
```

```
Plot_circle_points(xc, yc, x, y)
while x < y:
    x += 1
    if p < 0:
        p = p + 4 * x + 6
    else:
        y -= 1
        p = p + 4 * (x - y) + 10
    Plot_circle_points(xc, yc, x, y)
    return points.
xc = int(input("Enter x-coordinate: "))
yc = int(input("Enter y-coordinate: "))
x = int(input("Enter radius:"))

circle_points = bresenham_circle(xc, yc, r)

Print("Generated circle points: ")
for p in circle_points:
    Print(p)

x_values_circle = [p[0] for p in circle_points]
y_values_circle = [p[1] for p in circle_points]

Plt.figure(figsize = (8, 8))
Plt.plot(x_values_circle, y_values_circle, 'o')
Plt.title("Bresenham's Circle Drawing Algorithm")
Plt.xlabel("x-axis")
```

```
Plt.ylabel("y-axis")
Plt.grid(True)
Plt.gca().set_aspect('equal', adjustable='box')
Plt.show()
```

RESULT :

Hence implemented Bresenham's line Algorithm for circle using python Programme.

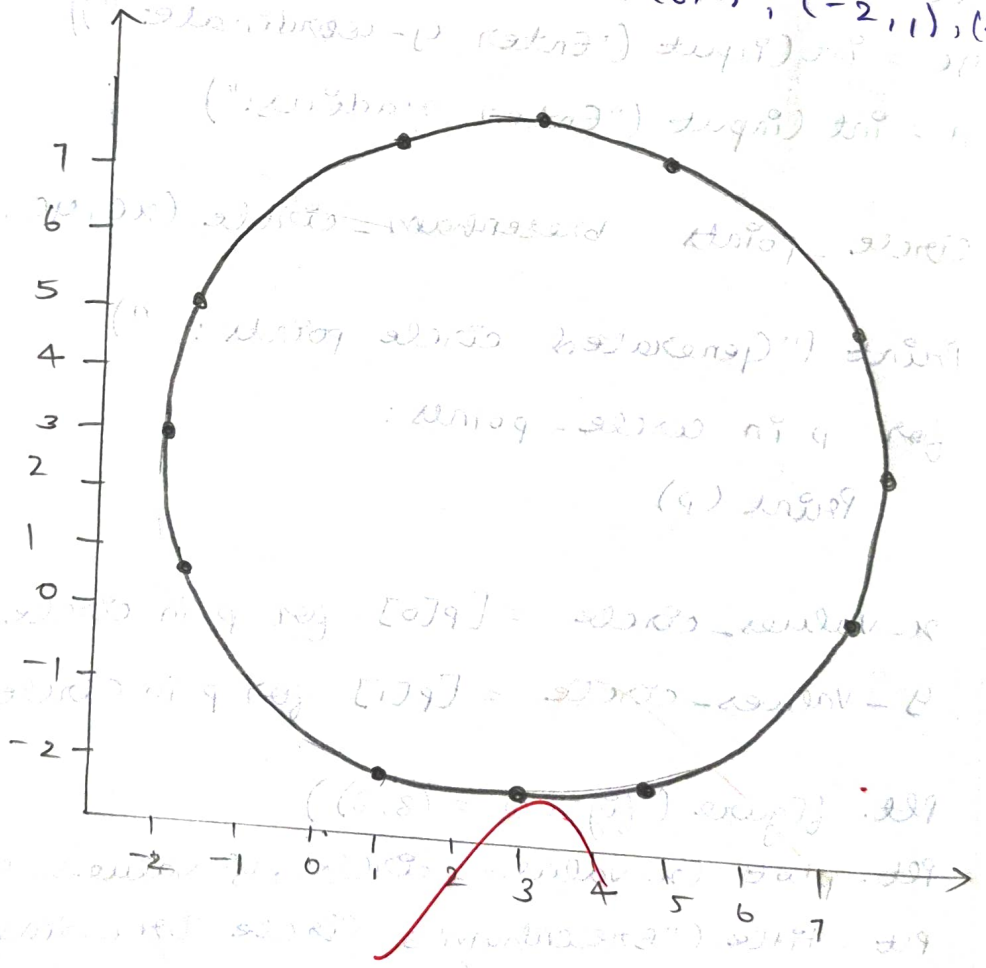
INPUT :

Enter the x-coordinate of circle's centre (xc) : 2

Enter the y-coordinate of circle's centre (yc) : 2

Enter the radius of the circle (r) : 4

Points: (2,6), (2,6), (2,-2), (2,-2), (6,2), (-2,2), (6,2), (-2,2), (6,2), (-2,2), (3,6), (1,6), (3,-2), (6,3), (-2,3), (6,1), (-2,1), (4,5)



21/7/26

EXPERIMENT - 3

BRESENHAM'S LINE DRAWING ALGORITHM FOR ECLIPSE

AIM :

To implement Bresenham's Line Drawing algorithm for eclipse using Python Programming.

PROGRAM :

```
import matplotlib.pyplot as plt
import math.

def bresenham_ellipse(xc, yc, rx, ry):
    unique_points = set()

def plot_ellipse_points(cx, cy, px, py):
    unique_points.add((cx+px, cy+py))
    unique_points.add((cx-px, cy+py))
    unique_points.add((cx+px, cy-py))
    unique_points.add((cx-py, cy-py))

    x = 0
    y = ry
```

```
p1 = round(ry * ry - rx * rx * ry + rx * rx / 4)
dx = 2 * ry * ry * x
dy = 2 * rx * rx * y
while dx < dy:
    plot_ellipse_points(xc, yc, x, y)
    x += 1
    dx = dx + 2 * ry * ry
    if p1 < 0:
        p1 = p1 + dx + ry * ry
    else:
        y -= 1
        dy = dy - 2 * rx * rx
        p1 = p1 + dx - dy + ry * ry
points = list(unique_points)
points.sort(key=lambda p: math.atan2))
return points.

xc_e = int(input("Enter xc: "))
yc_e = int(input("Enter yc: "))
rx_e = int(input("Enter rx: "))
ry_e = int(input("Enter ry: "))
ellipse_points = bresenham_ellipse
                 (xc_e, yc_e, rx_e, ry_e)
```

```
Print ("Generated ellipse points : ")
for p in ellipse_points :
    print (p)

Plt. figure (figsize = (8, 8))
Plt . xlabel ("x. axis")
Plt. ylabel ("y. axis)
Plt. grid (True)
Plt. show ()
```

RESULT :

Hence implemented Bresenham's Line Drawing Algorithm for Ellipse using Python Programming.

INPUT :

Enter x-coordinate of ellipse centre

$xc = 50$

Enter y-coordinate of ellipse centre

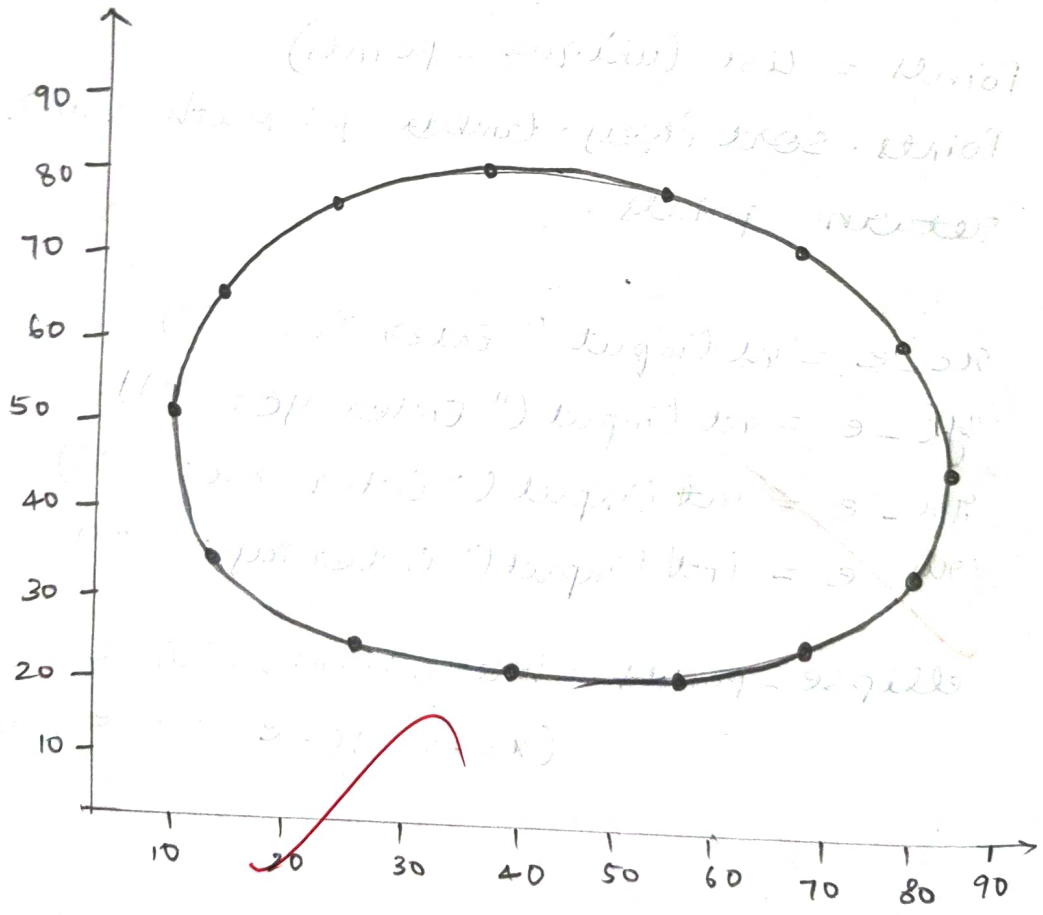
$yc = 50$

Enter major radius of ellipse

$rx = 40$

Enter minor radius of ellipse

$ry = 20$.



04/08/26

EXPERIMENT- 4

IMPLEMENTATION OF DDA LINE DRAWING ALGORITHM.

AIM :

To implement the Digital Differential Analyser (DDA) Line Drawing Algorithm in Python.

PROGRAM :

```
import matplotlib.pyplot as plt

def dda_line_drawing(x0, y0, x1, y1):

    dx = x1 - x0
    dy = y1 - y0

    steps = abs(dx) if abs(dx) > abs(dy)
                    else abs(dy)

    x_inc = dx / float(steps)
    y_inc = dy / float(steps)

    x = x0
    y = y0

    Points = []

    Points.append((round(x), round(y)))
```

```
for _ in range (int (steps)):
    x += x_inc
    y += y_inc
    points.append ((round (x), round (y)))
return points.

x0_short, y0_short = 0, 0
x1_short, y1_short = 10, 7

line_points_short = dda_line (x0_short, y0_short
                              x1_short, y1_short)

Print (f" points generated by DDA algorithm:
          {line_points_short} \n")

Plt.grid (True)
Plt.show ()
```

4/8/26

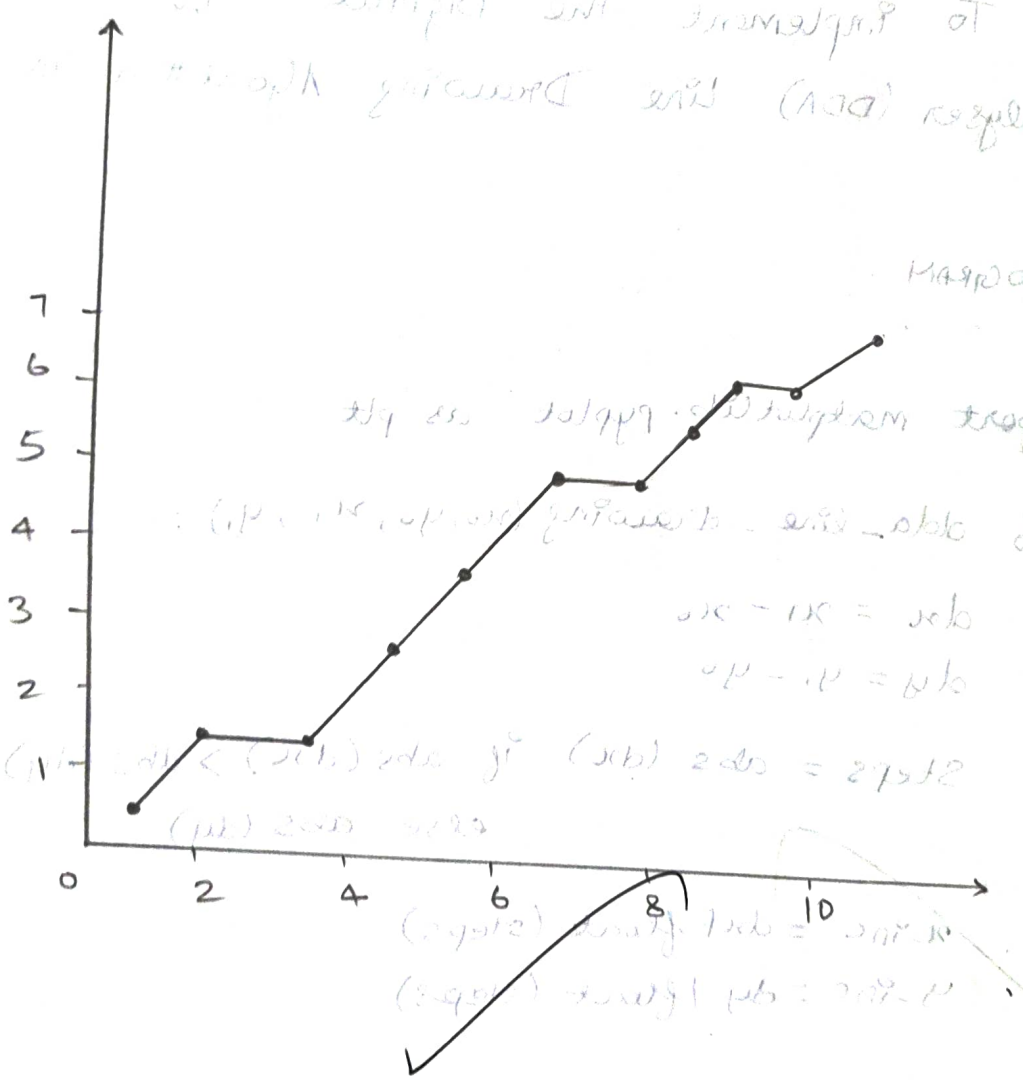
RESULT :

Hence implemented DDA algorithm using python programming.

OUTPUT :

Points generated :

[(0,0), (1,1), (2,1), (3,2), (4,3), (5,4), (6,4),
(7,5), (8,6), (9,6), (10,7)].



11/8/24

EXPERIMENT : 6

DRAW A RECTANGLE USING BRESENHAM'S ALGORITHM.

AIM :

To draw the rectangle using bresenham's algorithm from top to left (2, 2) with width = 8, height = 5.

PROGRAM :

```
import matplotlib.pyplot as plt
def bresenham(x0, y0, x1, y1):
    P = [];
    dx = abs(x1, -x0);
    dy = abs(y1 - y0)
    Sx = 1 if x0 < x1 else -1;
    e = dx - dy
    while 1:
        P.append((x0, y0))
        if (x0, y0) == (x1, y1): return p
        e2 = 2 * e
```

```
if e2 > -dy :  e -= dy ; x0 += sx
if e2 < dx :   e += dx ;  y0 += sy

x, y, w, h = 2, 2, 8, 5
xe, ye = x+w-1 , y+h-1

for a, b, c, d in [(x, y, xe, y) , (x, ye, xe, ye),
                   (x, y, x, ye), (xe, y, xe, ye)]:
    P = bresenham (a, b, c, d)
    Plt . plot (* zip (*P ), 'ro-')

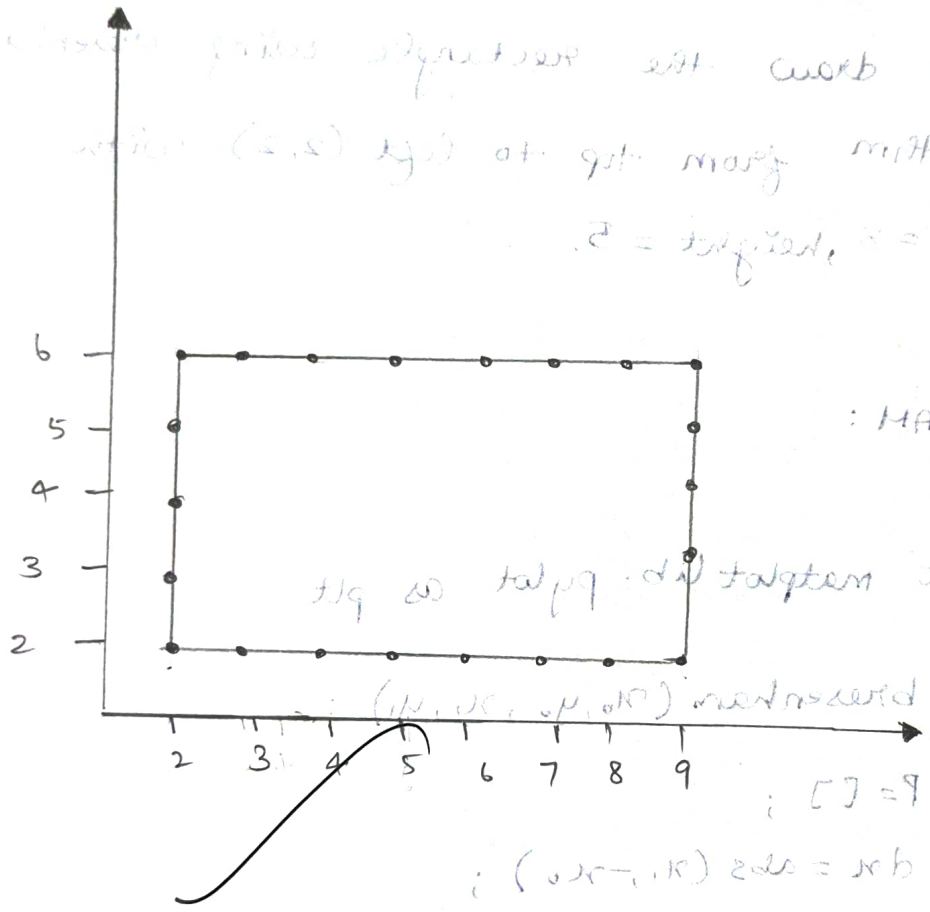
Plt . grid ()
Plt . axis ('equal')
Plt . show ()
```

11/8/24

RESULT :

Hence drawn the required rectangle and satisfy the necessary conditions.

OUTPUT :



11/8/26

EXPERIMENT : 7

DRAW A TRAFFIC SIGNAL USING BRESENHAM'S ALGORITHM.

AIM:

To draw a traffic signal following bresenham's line algorithm.

PROGRAM:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Rectangle
fig, ax = plt.subplots()
box = Rectangle((2,1), 4, 10, facecolour = "black")
ax.add_patch(box)
red = Circle((4,8), 1, facecolour = "red",
             edge colour = "white")
add_patch(red.
```

```
yellow = circle((4,6), 1, facecolor="yellow",
                edgecolor="white")
ax.add_patch(yellow)

green = circle((4,4), 1, facecolor="green",
               edgecolor="white")
ax.add_patch(green)

ax.set_xlim(0,8)
ax.set_ylim(0,10)
ax.set_aspect("equal")
ax.grid(True)

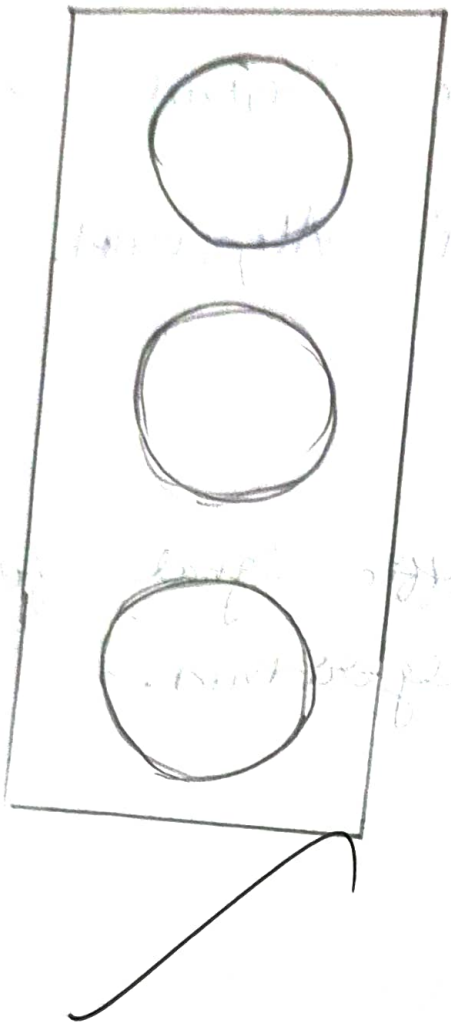
plt.title("Traffic Signal")
plt.show()
```

11/8/26

RESULT :

Hence implemented the traffic signal using python programming.

OUTPUT :



25.08.26.

EXPERIMENT : 9

DRAW THE OLYMPICS SYMBOL USING BRESENHAM'S ALGORITHM.

AIM :

To draw the olympics symbol using Bresenham's algorithm in python programming.

PROGRAM :

```
import matplotlib.pyplot as p
import numpy as np.

def C (x,y,r ,col):
    a = []; x,Y,d = 0, r, 3- 2 * r
    while x <= Y :
        a+ = [(x+X,y+Y), (x-X,y+Y), (x+X,y-Y), (x-X,y-Y),
              (x+Y, y+X), (x-Y,y+X), (x+Y, y-X), (x-Y,y-X)]
        if d<0 : d+ = 4*X +6.
        else : d+ = 4 * (x-Y) +10; Y -=1
        X+ =1
```

```
a.sort (key = lambda q: np.arctan2(q[1] - y, q[0] - x))
a += [a[0]]
p.plot (*zip (*a), colour = col, linewidth = 3)
p.scatter (*zip (*a[:-1]), color = col, s = 3)

for x, y, col in [(50,55,'blue'), (100,55,'black'),
                  (150,55,'red'), (75,30,'yellow'),
                  (125,30,'green')]:

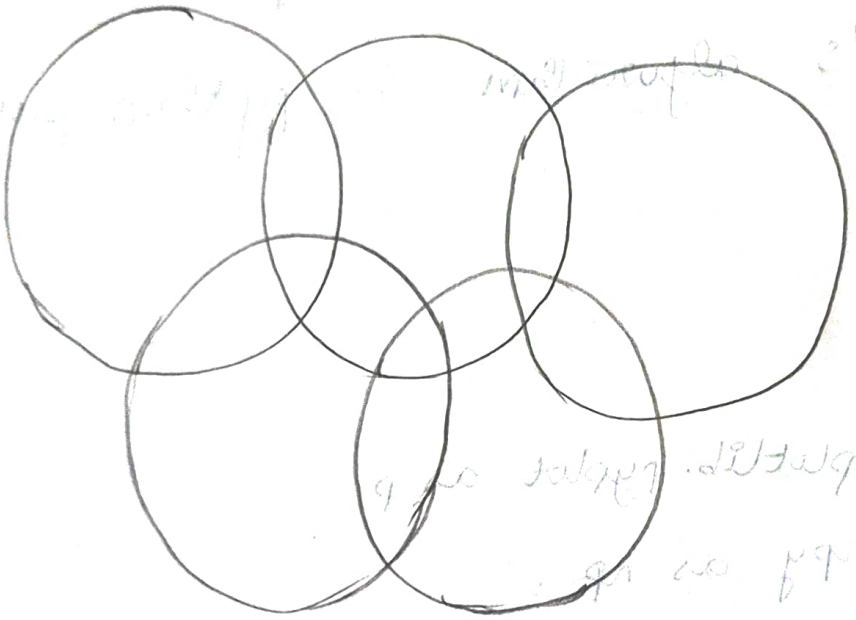
    c(x,y,25,col)

p.axis('equal')
p.axis('off')
p.show().
```

RESULT:

Hence drawn the olympics logo using Bresenham's algorithm in python programming.

OUTPUT :



8.9.26

EXPERIMENT : 11

IMPLEMENT SCAN CONVERSION OF POLYGON AND USE FLOOD FILL ALGORITHM.

AIM:

To implement scan conversion of polygon and use flood fill algorithm.

PROGRAM:

```
import matplotlib.pyplot as plt
P= [(2,2), (5,8), (9,3), (7,1), (3,1)]
def scan_fill(p):
    ys = range(min(y for x,y in p), max(y for x,y
                                        in p) +1)
    for y in ys:
        x=[]
        for i in range (len (p)):
            x1, y1 = p[i]
            x2, y2 =P [(i+1) %. len(p)]
            if y1 != y2 and min(y1, y2) <=y
                x. append (x1 + (y-y1) * (x2 - x1)/
                           (y2-y1))
            x. sort()
```

```
for i in range (0, len (x), 2):
plt. plot (x[i:i+2], [y,y], 'b-')

x,y = zip (*(p+[p[0]]))
plt. plot (x,y, 'k-')

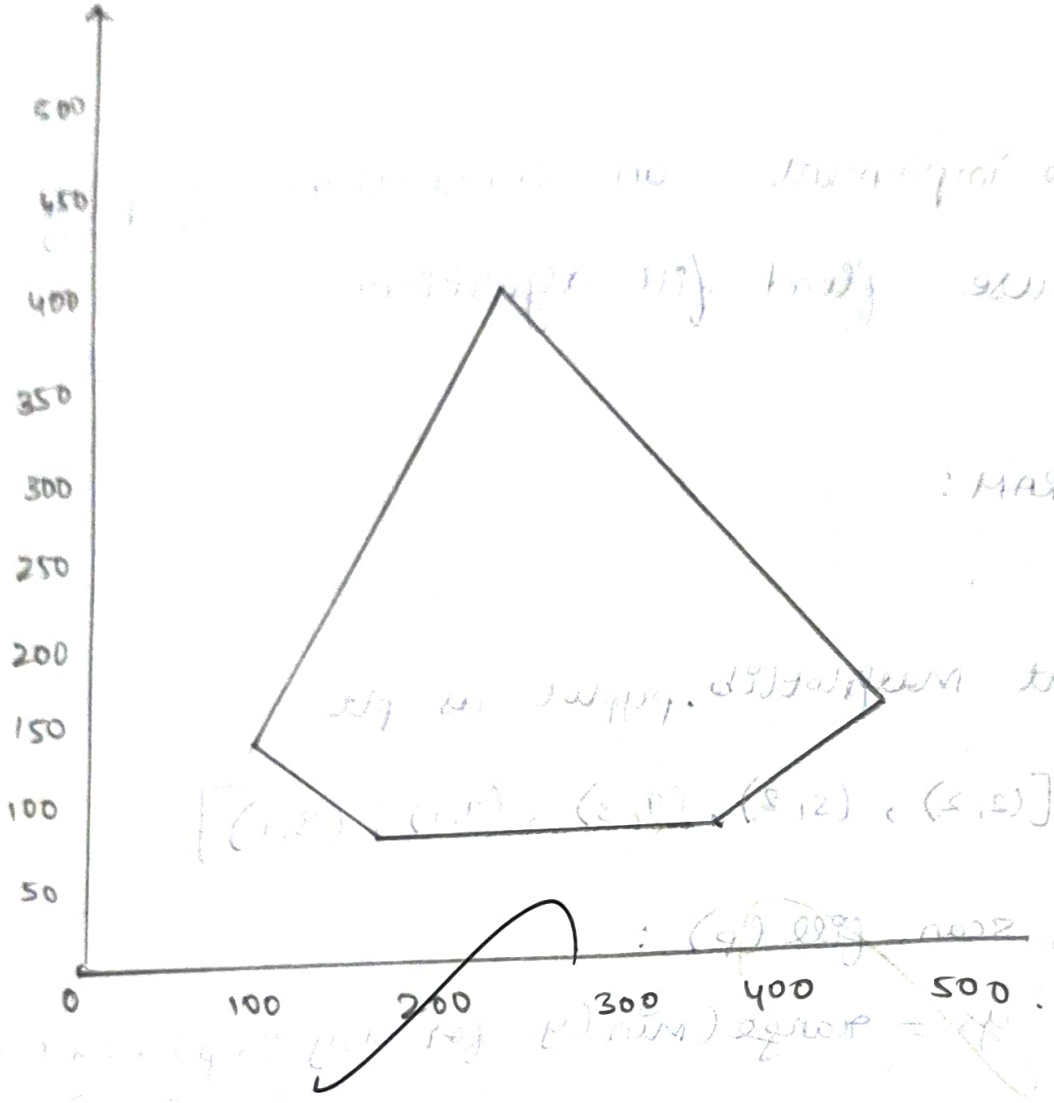
scan-fill (p)

plt. fill (x,y, color = 'lightblue', alpha = 0.5)
plt. axis ('equal')
plt. title ("Scan conversion and flood fill")
plt. show ().
```

RESULT :

Hence Implemented scan conversion of Polygon and use flood fill algorithm.

OUTPUT:



EXPERIMENT - 12

2D TRANSFORMATION

TRIANGLE.

AIM:

To implement 2D Transformation in a triangle. using a python program.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt.

original_shape = np.array([
  [0,0], [2,0], [1,2], [0,0] ])

def plot_shape (ax, shape, color, label):
  ax.plot (shape [:, 0], shape [:,1], color=color,
    label = label, marker = 'o')
  ax.fill (shape [:,0], shape [:,1], color=color,
    alpha = 0.3)
  ax.set_aspect ('equal', adjustable = 'box')
```

```
def translate (shape, dx, dy):
    translation_vector = np.array ([dx, dy])
    return shape + translation_vector.

def scale (shape, sx, sy, pivot = (0, 0)):
    scaling_matrix = np.array ([
        [sx, 0],
        [0, sy] ])
    translated_shape = shape - np.array(pivot)
    scaled_shape = translated_shape.

dx, dy = 2, 1
translated_shape = translate (original_shape,
                              dx, dy)
fig, ax = plt.subplots (figsize = (6, 6))
plot_shape (ax, original_shape, 'blue', 'original shape')
ax.legend ()
plt.show().

angle = 45
Pivot_point = (0, 0)
rotated_shape = rotate (original_shape, angle,
                        pivot = pivot_pivot).
```

```
ax.set_title('2D Rotation')
ax.legend()
plt.show().

sx, sy = 2, 0.5
pivot_point = (0,0)
scaled_shape = scale(original_shape, sx, sy)

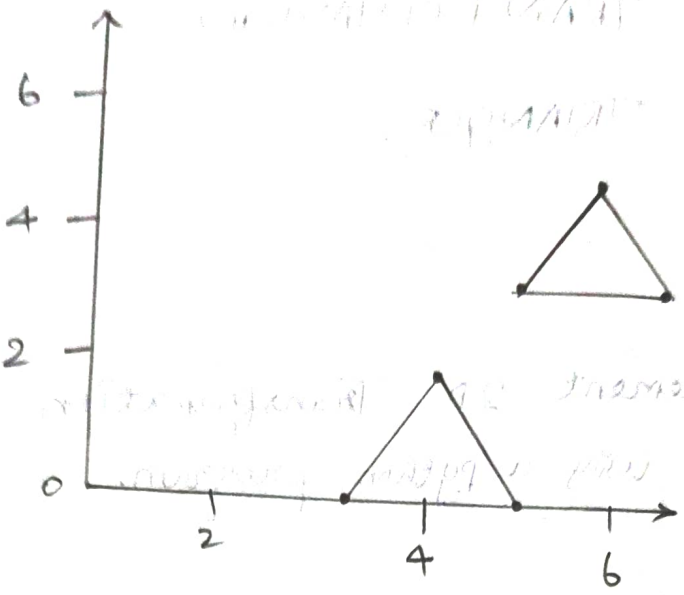
fig, ax = plt.subplots(figsize = (6,6))
ax.set_title('2D Scaling')
ax.legend()
plt.show().
```

22/9/26

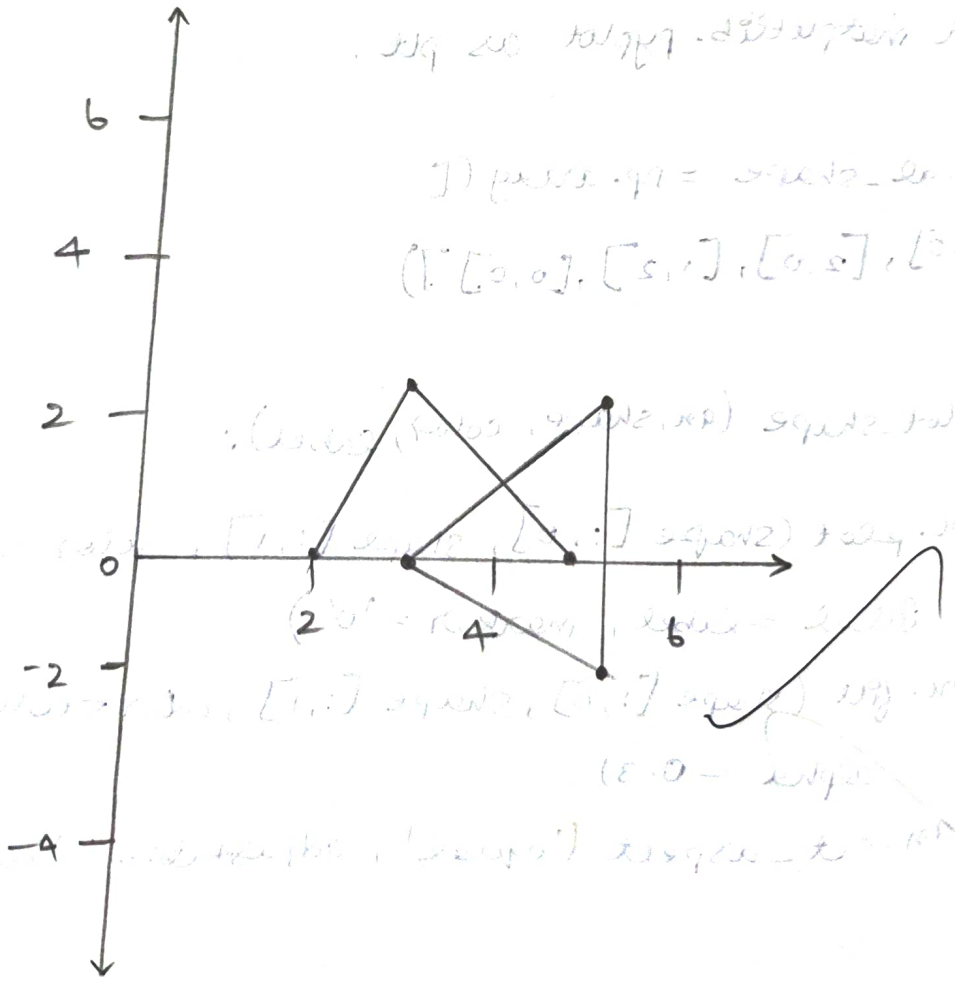
RESULT:

Hence implemented 2D transformation on a triangle using python programming.

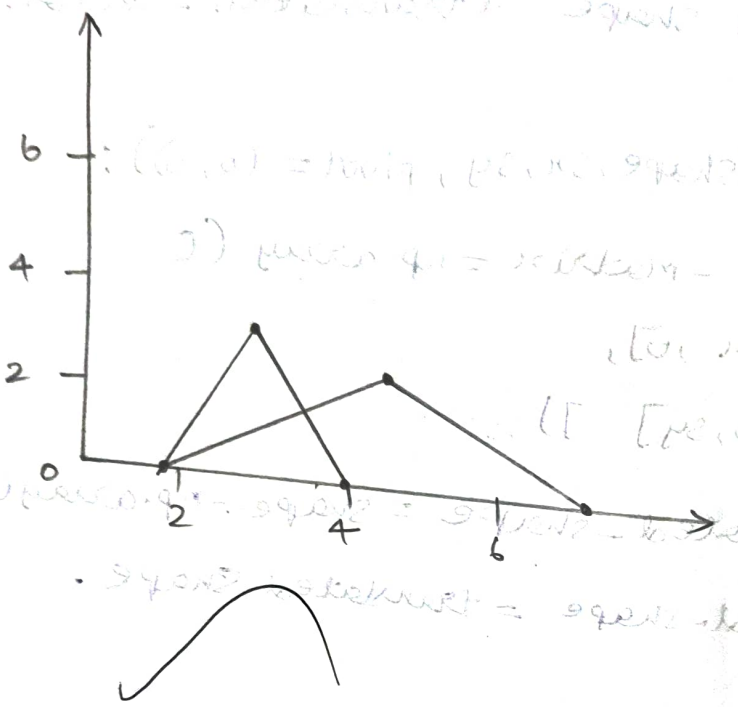
2D TRANSLATION



2D ROTATION



2D SCALING



EXPERIMENT : 13.

2D TRANSFORMATION IN RECTANGLE.

AIM :

To implement 2D transformation in rectangle using python program.

PROGRAM :

```
original_shape =np.array ([
[0,0], [3,0] , [3,2], [0,2], [0,0]
])

dx, dy = 1, 2.5
translated_ rect = translate (original_shape , dx ,dy)

fig, ax =plt. subplots (figsize = (8,8))
Plot_shape (ax , original_shape _rect, 'blue')
Plot-Shape (ax , translated-rect , 'red')
ax. legend()
Plot. show ().
```

```
angle = 30
pivot_point = (0,0)
rotated_rect = rotate (original_shape, angle,
                       pivot = pivot_pivot)

fig, ax = plt.subplots (figsize = (6,8))
ax.set_title ('2D Rotation (Rectangle)').
ax.legend()
plot.show().
```

```
sx, sy = 0.5, 1.5
pivot_point = (0,0)
scaled_rect = scale (original_shape, sx, sy,
                     pivot = pivot)

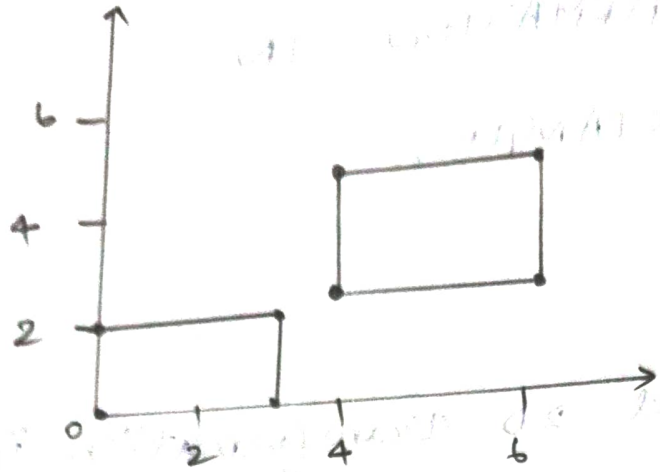
fig, ax = plt.subplots (figsize = (6,6))
plot_shape (ax, original_shape, 'blue').
ax.set_title ('2D Scaling (Rectangle)')
ax.legend()
Plot.show().
```

22/9/25

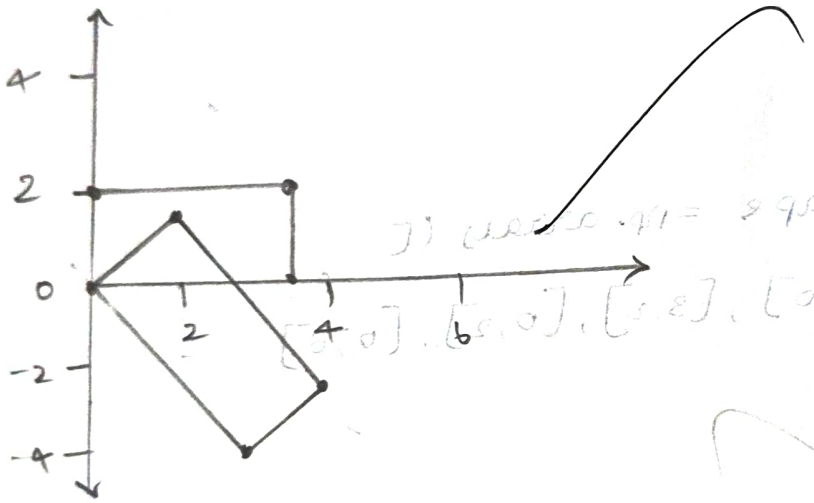
RESULT :

Hence implemented 2D transformation on a rectangle using python.

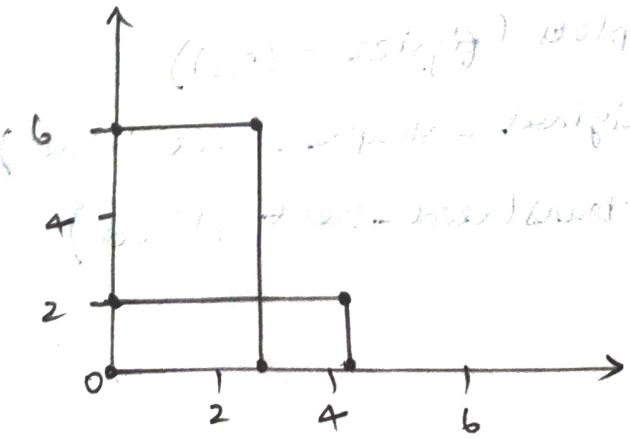
2D TRANSLATION



2D ROTATION



2D SCALING



EXPERIMENT : 14.

POLYGON CLIPPING.

AIM:

To implement Sutherland - Hodgman polygon clipping on a polygon.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt.

def inside (p, cp1, cp2):
    clip-edge_vec = cp2 - cp1
    point_vec = p - cp1
    return (clip-edge_vec [0] * point_vec [1]) -
        (clip-edge [1] * * point_vec (0]) >=0.

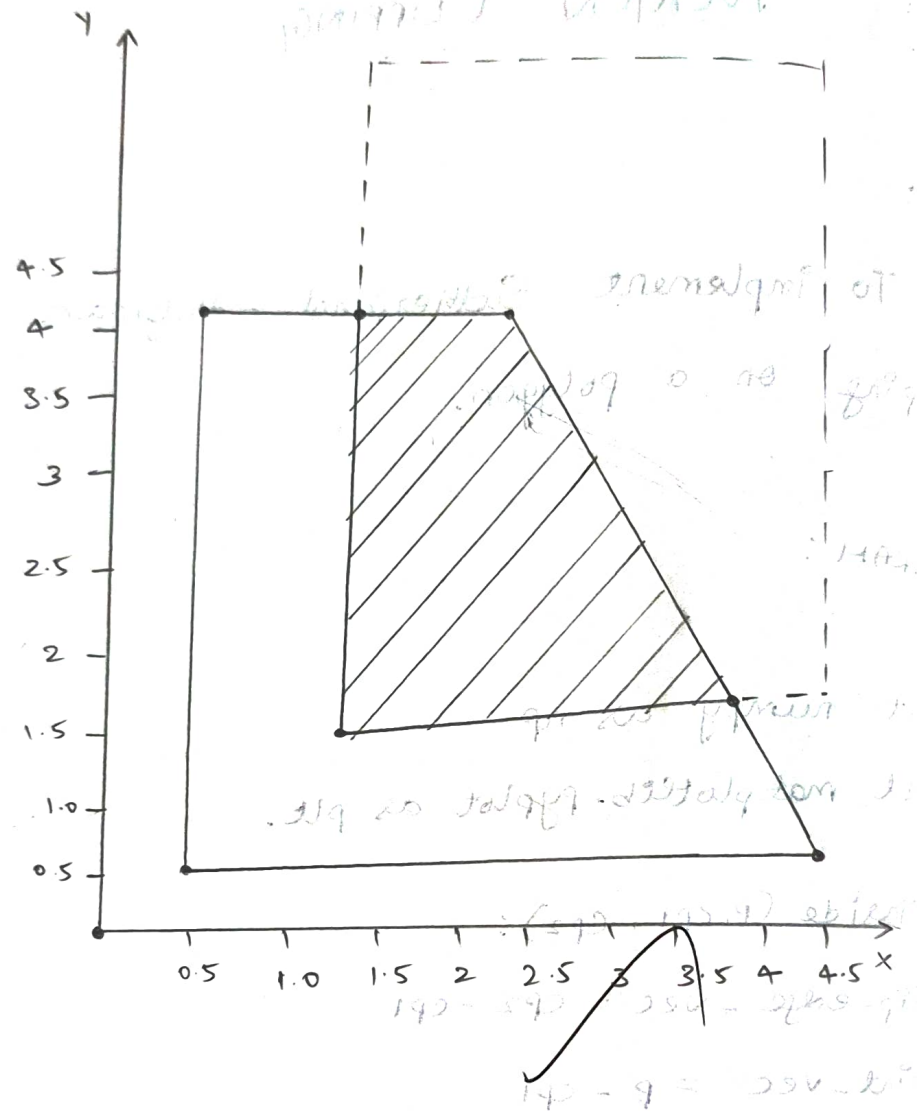
def sutherland_hod (subject_polygon, clip-polygon):
    output-polygon (list (subject_polygon).
```

```
for i in range (len (clip-polygon)-1):
    cp1 = clip-polygon[i]
    cp2 = clip-polygon[i+1]
    input_poly = output_poly
    output_poly = []
if not input_polygon :
    break
for j in range (len (input_poly)) :
    P1 = input_poly [j-1]
    P2 = input_poly [j]
    P2_inside = inside (P2, cp1, cp2)
    P1_inside = inside (P1, cp1, cp2).
if P2_inside :
    if not P1_inside :
        intersection_pt := iderect (P1, P2, cp1, cp2).
return np.array (output_poly)
ax.grid (True)
ax.legend ()
plt.show ()
```

RESULT:

Hence implemented sutherland-Hodgman polygon clipping.

OUTPUT:



EXPERIMENT : 15

3D TRANSFORMATION IN CUBE.

AIM:

To Implement 3D transformation in Cube.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D.

def create_translation_matrix(tx, ty, tz):
    return np.array([[1,0,0,tx], [0,1,0,ty],
        [0,0,1,tz], [0,0,0,1]])

def create_scaling_matrix(sx, sy, sz):
    return np.array([[sx,0,0,0], [0,sy,0,0],
        [0,0,sz,0], [0,0,0,1]])

original_cube_vertices = np.array(
[[0,0,0], [1,0,0], [0,1,0], [0,0,1], [1,1,0],
[1,0,1], [0,1,1], [1,1,1]].
```

```
def plot_cube (ax, vertices, edge_color, label,
               face_color = None, alpha = 1.0):

edges = [ (0,1), (0,2), (0,3), (1,4), (1,5), (2,4), (2,6),
          (3,5), (3,6), (4,7), (5,7), (6,7) ]

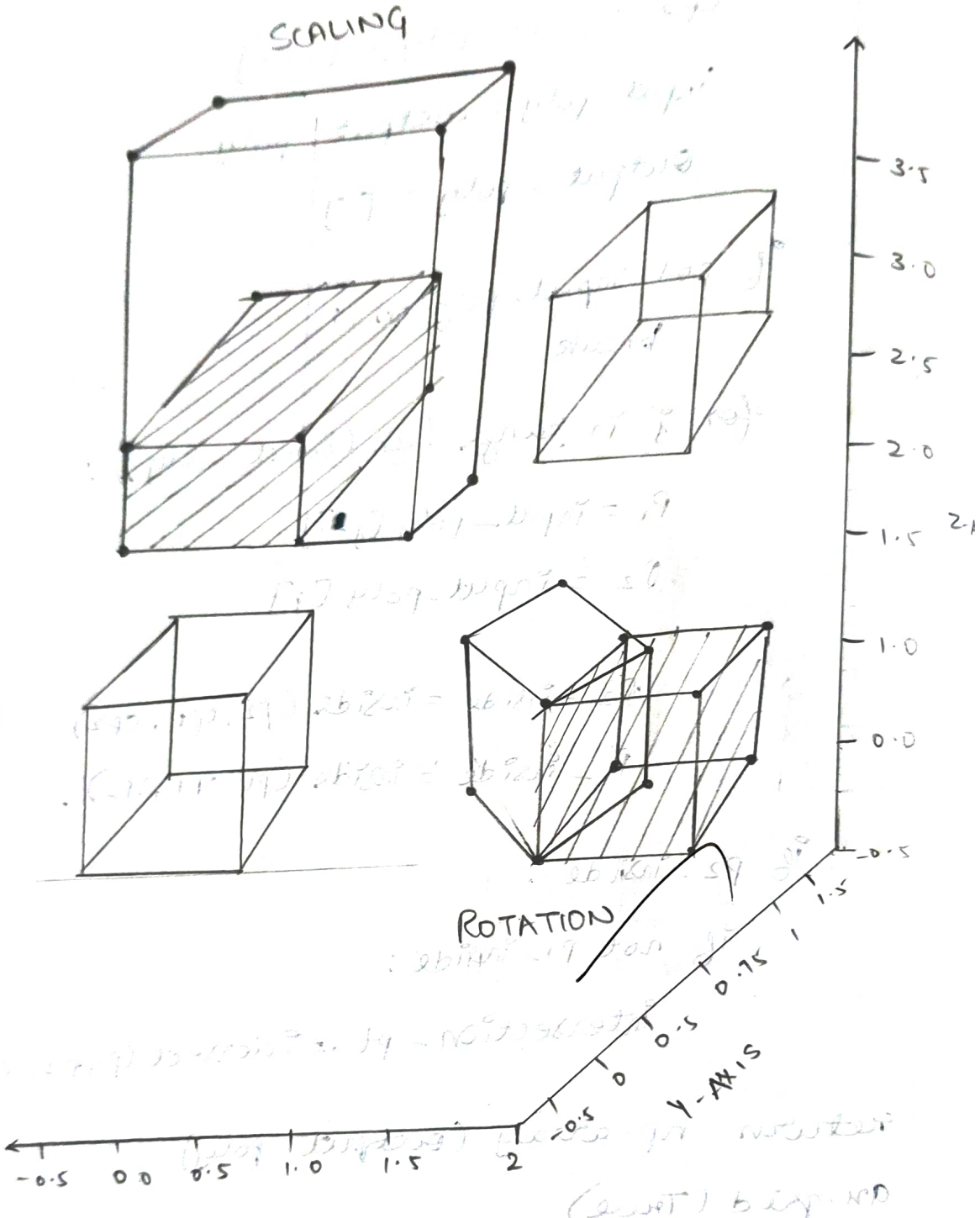

for i, j in edges:
    ax.plot (* zip (vertices[i], vertices[j]),
             color = edge_color)

if face_color:
    poly_collection = Poly3DCollection (faces,
      alpha = alpha, facecolor = face_color,
      edge_color = 'none').
    ax.add_collect (poly_collections).

ax.set_xlabel ('x-axis')
ax.set_ylabel ('y-axis').
ax.set_zlabel ('z-axis')
ax.set_title (title)
ax.legend ().
plt.show ()
```

RESULT :

Hence implemented 3D transformation in a cube.


SCALING
3.5
3.0
2.5
2.0
1.5
2.1
1.0
0.0
-0.5
ROTATION
-0.5
0.0
0.5
1.0
1.5
2
X-AXIS
-0.5
0
0.5
0.75
1
1.5
Y-AXIS

EXPERIMENT : 18.

CREATE A MULTIMEDIA APPLICATION TAT INTEGRATES IMAGES, SOUND AND VIDEO IN A SIMPLE UI

AIM:

To develop a simple multimedia application that integrates audio, video, Images. into a unified UI using python.

PROGRAM:

```
import tkinter as tk
from tkinter import filedialog
import cv2.
import threading.

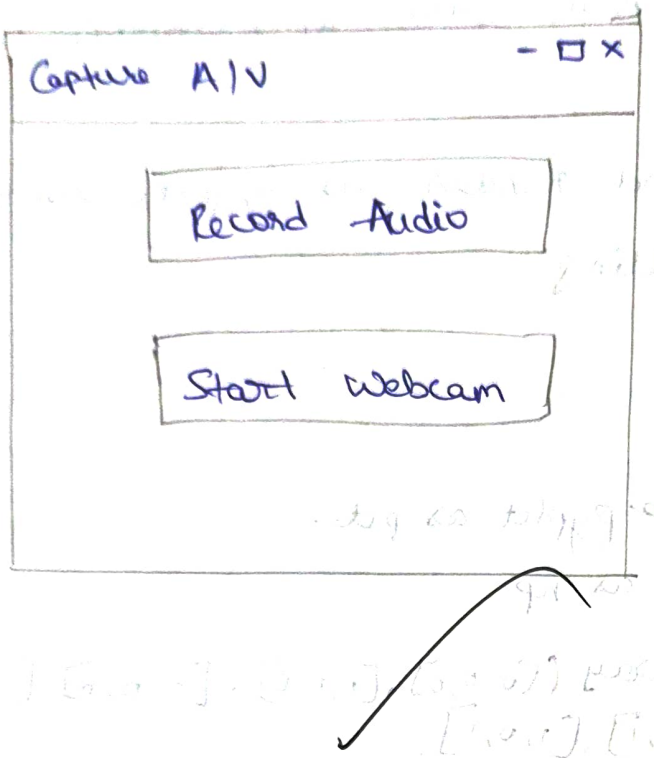
def play_audio():
        Playsound ('sample_audio.mp3')
def play_vido():
    cap = cv2.VideoCapture ('sample_vido.mp4').
def load_image():
    img = Image.open ('sample_image.jpg')

window = tk.TK()
Panel = tk.label (window).
Panel.pack().
windao.mainloop().
```

RESULT:

A multimedia application was successfully Created.

OUTPUT :



EXPERIMENT :19.

CAPTURE VIDEO / AUDIO FROM WEBCAM / MICROPHONE.

AIM:
To develop a program that captures like audio / video and displays them.

PROGRAM.

```
import cv2
import sounddevice as sd
from scipy.io wavfile import write.

def audio_rec():
    duration = 5
    fs = 44100
    sd.wait.

def vdo_cap():
    cp = cv2.VideoCapture(0)
    cap.release()
    cv2.destroy()

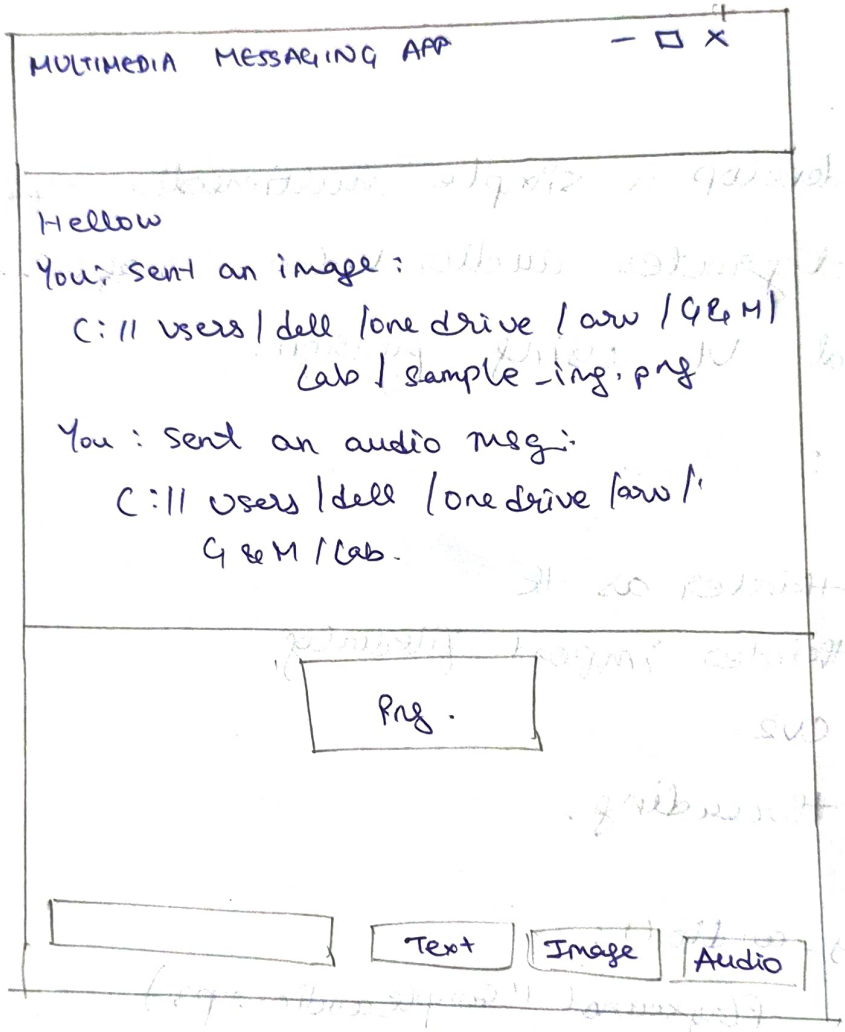
window = tk.Tk()
btn_audio.pack(pady = 10).
btn_video.pack(pady = 10)

window.mainloop().
```

RESULT :

Live audio / video were sucessfully captured.

OUTPUT :


MULTIMEDIA MESSAGING APP
Hellow
You: sent an image:
C:\\ Users \ dell \ one drive \ aru \ G&M \
Lab \ sample_img.png
You: sent an audio msg:
C:\\ Users \ dell \ one drive \ aru \
G&M \ Lab.
Png.
Text
Image
Audio

Completed

مرحلة